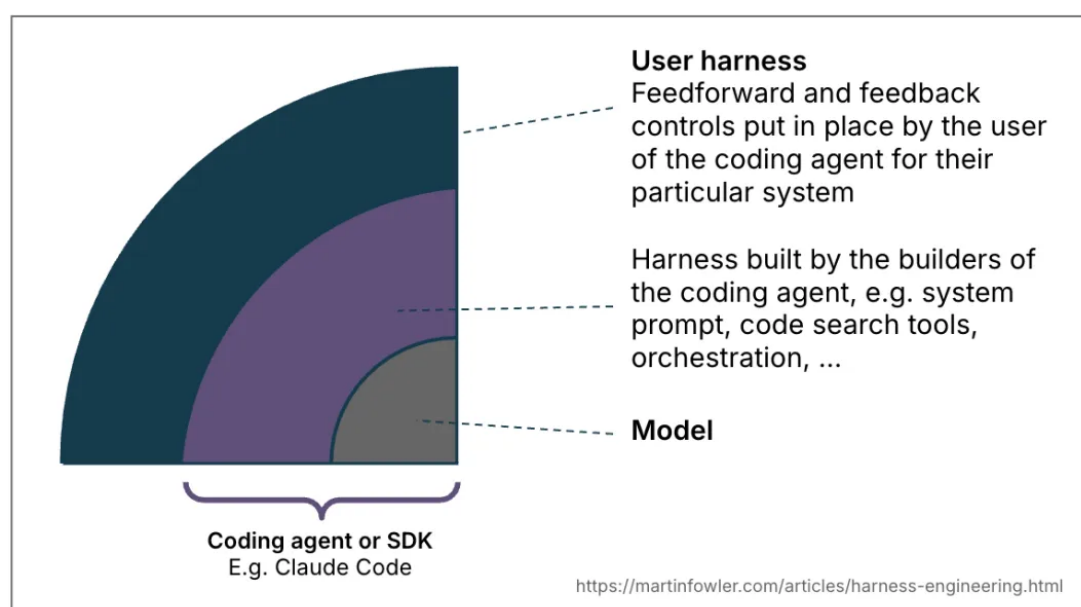


AI Coding 用户的 Harness Engineering 指南

要让 Coding Agent 在更少人工监督下工作，我们需要建立对其输出结果的信任机制。作为软件工程师，我们对 AI 生成的代码天然存在信任壁垒——LLM 是非确定性的，它不了解我们的上下文，也不真正「理解」代码，它以 token 为单位思考。本文探讨一种将上下文工程与 Harness Engineering 新兴概念融合在一起的思维模型，用以建立这种信任。

「Harness」这个词已经成为一个简称，泛指 AI agent 中除模型本身之外的一切——**Agent = Model + Harness**。这个定义范围很广，因此有必要针对具体类别的 agent 加以收窄。本文专注于「Coding Agent」这一语境下的定义。Coding Agent 本身已内置了部分 harness（例如系统提示、代码检索机制，甚至复杂的编排系统）。但 Coding Agent 也为用户提供了大量功能，让我们能为自己的具体用例和系统构建一套「外层 harness」。

一套构建良好的外层 harness 服务于两个目标：**提高 agent 第一次就做对的概率**，并提供一个反馈回路，在问题到达人眼之前尽可能自我纠正。最终效果是减少代码审查的负担、提升系统质量，同时附带减少 token 浪费的好处。



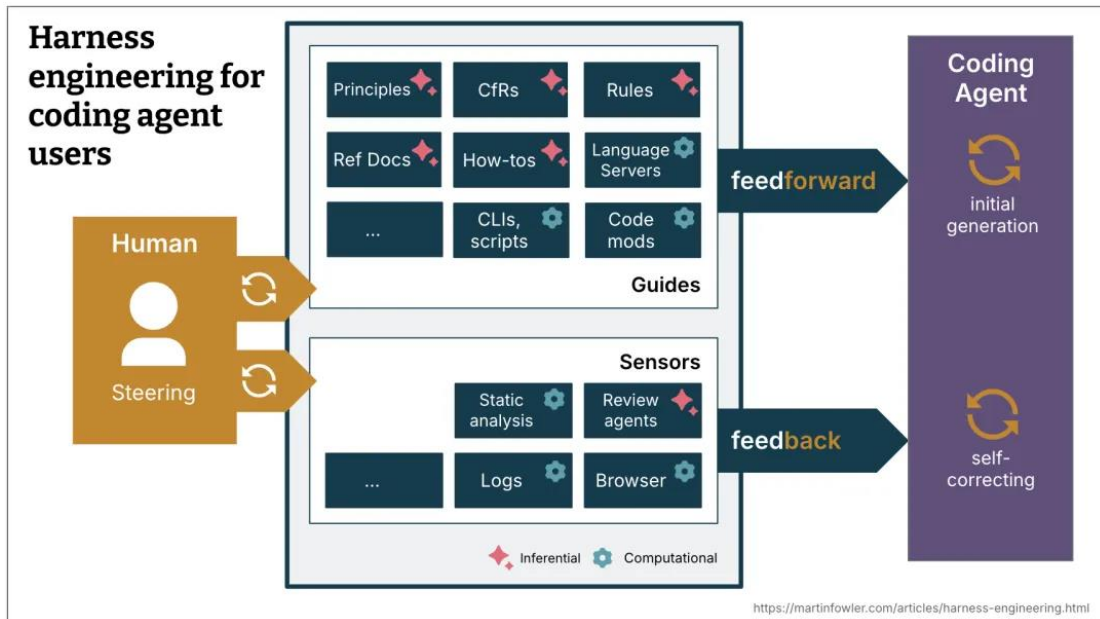
一、前馈与反馈：Harness 的两根支柱

要驾驭一个 AI Coding Agent，我们既要预判不良输出并加以预防，也要部署传感器让 agent 能够自我修正：

Guides（引导机制，前馈控制）——预判 agent 的行为，在它行动之前加以引导。Guides 提高 agent 第一次生成好结果的概率。

Sensors（感知机制，反馈控制）——在 agent 行动之后进行观察，帮助它自我修正。当 Sensors 产出针对 LLM 消费优化的信号时尤为强大——例如，自定义 linter 的报错信息中包含自我纠正的具体指令，这是一种正向的「提示注入」。

缺少任何一方，系统都会失效：只有反馈而没有前馈，agent 会反复犯同样的错误；只有前馈而没有反馈，agent 虽然写入了规则，却永远不知道规则是否真的生效了。



这个「前馈+反馈」的控制论框架，在工业控制系统领域已有数十年历史。Böckeler 把它引入 AI agent 工程，意义不仅仅是给概念贴标签——它揭示了一个根本问题：大多数团队目前对 coding agent 的使用停留在纯「前馈」模式（写好 prompt 就扔给 agent），缺乏系统性的反馈回路。没有反馈，agent 的行为边界完全依赖模型自我约束，而 LLM 的非确定性使这种约束极不可靠。

二、计算型 vs. 推理型：两种执行机制

Guides 和 Sensors 各自又分为两种执行类型：

计算型（Computational）——确定性强、速度快，由 CPU 执行。包括测试、linter、类型检查器、结构分析。运行时间从毫秒到秒级不等，结果可靠。

推理型（Inferential）——语义分析、AI 代码审查、「LLM as judge」。通常由 GPU 或 NPU 执行。速度较慢、成本较高，结果具有更强的非确定性。

计算型 Guides 通过确定性工具提高良好结果的概率。计算型 Sensors 足够廉价快速，可以随 agent 的每一次变更同步运行。推理型控制虽然更昂贵、更不确定，但允许我们提供更丰富的指导，并增加额外的语义判断层。尽管存在非确定性，推理型 Sensors 在配合强力模型（或者说适合当前任务的模型）使用时，依然能显著提升我们的信心。

下表来自原文，清晰展示了各类控制机制的分类和实现示例：

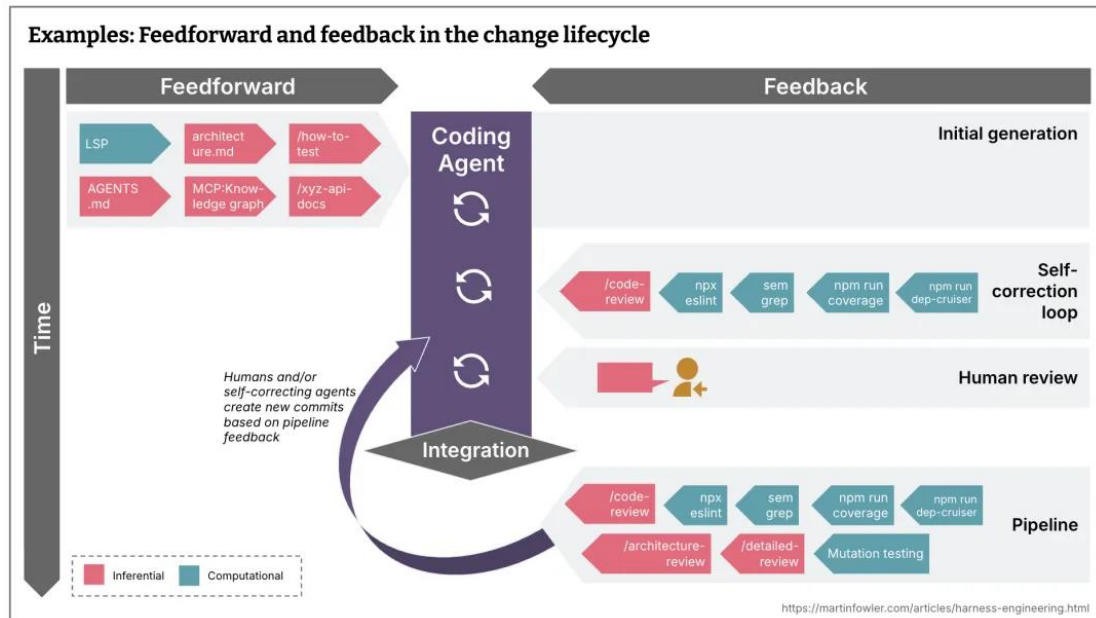
控制内容	方向	类型	实现示例
Agent 规范	前馈	推理型	AGENTS.md、Skills
新项目启动说明	前馈	两者皆有	含说明的 Skill + bootstrap 脚本
Code mods	前馈	计算型	接入 OpenRewrite recipes 的工具
结构测试	反馈	计算型	运行 ArchUnit 测试检查模块边界的 pre-commit hook
代码审查说明	反馈	推理型	Skills

计算型 vs. 推理型的分类，解决了一个实践中极易犯的错误：把本该用规则写死的约束，交给 LLM 去「理解和遵守」。这两件事的可靠性差了一个数量级。举一个具体例子：你在 AGENTS.md 里写「不要跨模块直接调用私有方法」，LLM 大多数时候会遵守，但偶尔会忘记或找理由绕过。而如果你把这个规则写成 ArchUnit 结构测试，违反就是编译失败——没有商量余地。能用计算型解决的，绝不依赖推理型。

三、时机：质量检查越靠前越好

持续集成实践者一直面临一个挑战：如何在开发时间线上合理分布测试、检查和人工审查，同时兼顾它们各自的成本、速度和重要性。理想状态下，每一次提交都应该是可部署的。原则是：**检查越靠近问题发生源头，修复成本越低。**

原文的生命周期示意图给出了一个实践框架：



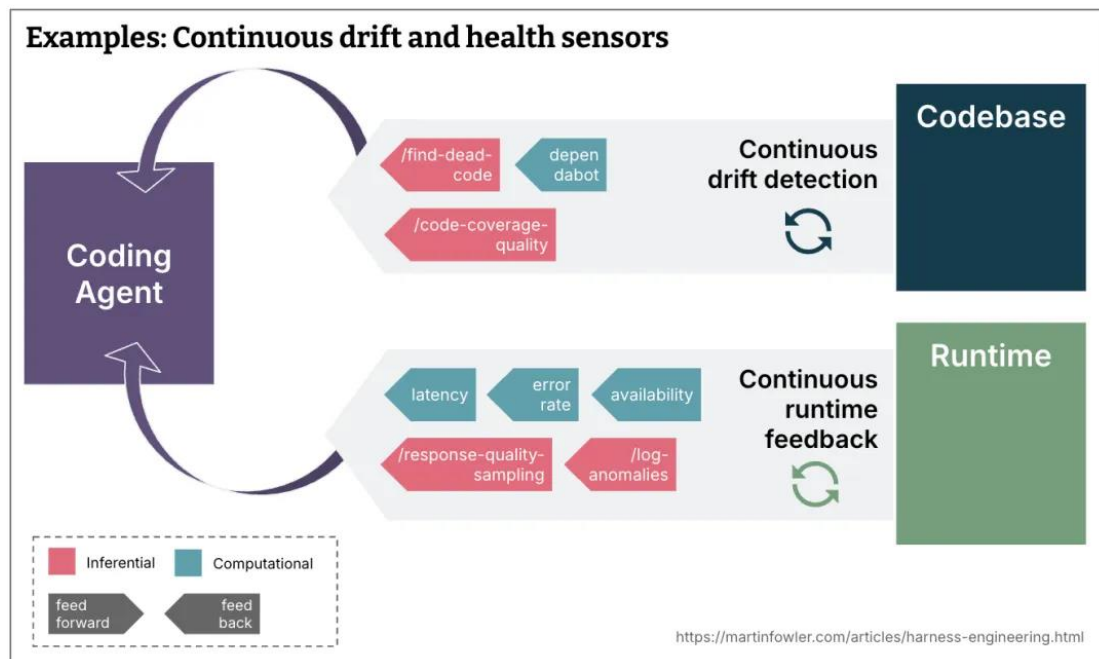
集成前甚至提交前：应运行哪些速度足够快的检查？（例如：linter、快速测试套件、基础代码审查 agent）

集成后流水线中：哪些更昂贵的检查只在此阶段运行，并重复执行快速检查？（例如：mutation testing、能考虑更大全局的广泛代码审查）

此外，还有两类持续运行的 Sensors 需要单独设计：

持续漂移检测——针对代码库持续运行，不依附于变更生命周期：如死代码检测、测试覆盖率质量分析、依赖扫描器（dependabot）。

持续运行时反馈——让 agent 监控运行时指标：如监测 SLO 退化并提出改进建议，AI judge 持续抽样响应质量并标记日志异常。



这里有一个微妙但重要的点：「持续漂移检测」打破了传统 CI/CD 的思维定式。传统上，检查都附着在「变更事件」上——有提交才有检查。但代码库的退化往往不是某一次提交造成的，而是长期无人处理的技术债慢慢积累的结果。设计一套独立于变更生命周期运行的 sensor 体系，是 harness 工程走向成熟的标志之一。OpenAI 和 Stripe 在各自的工程博客里都提到了类似的「垃圾回收」机制——这不是巧合。

四、Harness 的三个维度

Harness 并不是一个单一目标的系统，Böckeler 将其拆分为三个调节维度：

1. 可维护性 Harness (Maintainability)

目前最成熟的维度，因为已有大量预置工具可用。

计算型 Sensors 能可靠捕捉结构性问题：重复代码、圈复杂度、缺失的测试覆盖、架构漂移、风格违规。这些检查廉价、成熟、确定性强。

LLM 能部分解决需要语义判断的问题——语义层面的重复代码、冗余测试、暴力修复、过度设计——但成本较高且具有概率性，不适合在每次提交时运行。

而某些高影响力的问题，两者都无法可靠捕捉：问题诊断错误、过度工程化和不必要的功能、误解了人类的指令。这些问题有时能被发现，但不够可靠，无法减少监督需要。**正确性**是任何 sensor 都无法保证的，前提是人类一开始就没有清晰地说明自己想要什么。

2. 架构适应性 Harness (Architecture Fitness)

这是定义和检查应用架构特征的 Guides 和 Sensors 组合——本质上就是架构适应性函数 (Fitness Functions) 的概念。

示例：

向前馈送性能需求的 Skill，加上向 agent 反馈性能改善或退化的性能测试

描述更好可观测性 Agent 规范的 Skill（如日志标准），加上要求 agent 反思其所用日志质量的调试说明

3. 行为 Harness (Behaviour)

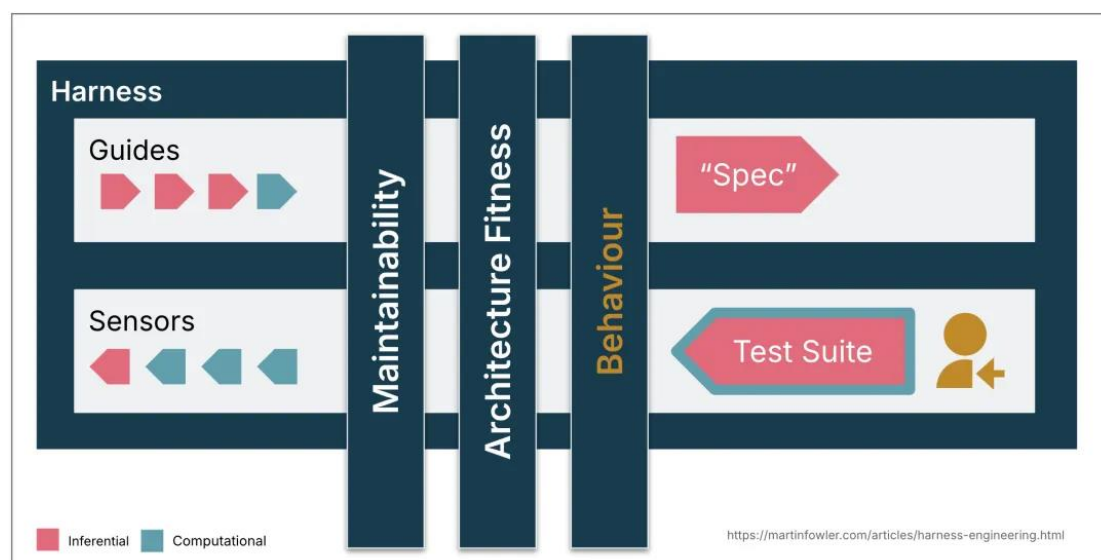
这是房间里的大象——我们如何引导和感知应用是否按预期功能运行？

当前实践中，给 agent 高自主权的团队通常这样做：

前馈：提供功能规格说明（详细程度不一，从简短 prompt 到多文件描述）

反馈：检查 AI 生成的测试套件是否通过，覆盖率是否合理，部分团队还会用 mutation testing 监控测试质量，并结合人工测试

这套方法对 AI 生成测试的信任度过高，目前还不够好。部分同事在使用「approved fixtures」模式时取得了不错的效果，但它更适合特定场景，并非对测试质量问题的全面解答。



行为 Harness 是整篇文章最诚实的部分。Böckeler 没有假装这个问题已经解决，而是直接承认：对于功能正确性，我们目前还没有好的系统性答案。这个坦诚很重要——它意味着当前所有声称「高自主度 coding agent」的方案，都在某种程度上依赖人工测试兜底。这不是技术局限，而是认知诚实。任何工程团队在这一层盲目信任 agent，都是在冒险。

五、Harness 能力 (Harnessability)

并非所有代码库都同样容易被 harness 化。

强类型语言天然提供类型检查作为 sensor；清晰可定义的模块边界提供架构约束规则；Spring 等框架屏蔽了 agent 无需操心的细节，从而隐性地提高了 agent 的成功率。缺乏这些特性，相应的控制机制就无从建立。

Böckeler 的同事 Ned Letcher 将代码库的这些特性称为「环境固有条件 (ambient affordances)」：「环境本身的结构特征，使其对在其中运行的 agent 来说更易读、更易导航、更易处理。」

这对新旧项目的影响截然不同：

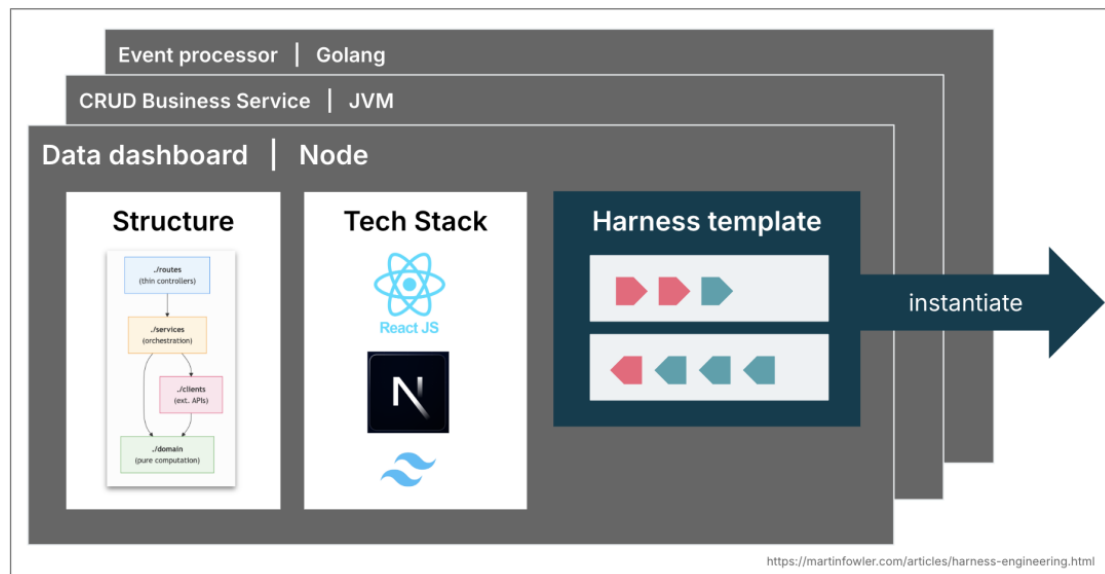
绿地项目：团队可以从第一天起就将 harnessability 内建进去——技术选型和架构决策决定了代码库未来的可管控程度。

遗留系统：尤其是积累了大量技术债的应用，面临的是更难的问题：harness 最需要的地方，往往是最难建立的地方。

「环境固有条件」这个概念值得展开思考。它预示着一个可能的趋势：未来技术选型的理由，将不再只是「哪个框架生产力最高」或「哪个语言性能最好」，而是会加上一个新维度——「哪套技术栈已经有成熟的 harness 可用」。正如 escape.tech 在旧金山的调研报告所指出的：「护城河在 harness，不在模型本身。」这意味着，率先为某个技术栈建立成熟 harness 体系的团队或工具，将在 agent 时代获得显著的先发优势。

六、Harness 模板：把经验变成基础设施

大多数企业都有少数几种常见的服务拓扑，覆盖 80% 的需求——暴露 API 的业务服务、事件处理服务、数据仪表盘。在成熟的工程组织中，这些拓扑往往已被 Coding Agent 成「服务模板」。



未来，这些可能演变为 **Harness 模板**：一套将 coding agent 绑定到特定拓扑结构、规范和技术栈的 Guides 和 Sensors 组合包。团队在技术选型和架构决策时，可能会部分基于「已有哪些现成 harness 可用」来做判断。

当然，这面临与服务模板类似的挑战：一旦团队从模板实例化出去，就开始与上游改进脱节。Harness 模板将面临同样的版本控制和贡献问题，而非确定性的 Guides 和 Sensors 更难测试，情况可能更糟。

七、人的角色：不可替代的隐性 Harness

作为人类开发者，我们将自己的技能和经验作为一种隐性 harness 带入每个代码库。我们吸收了规范和最佳实践，感受过复杂性带来的认知痛苦，知道 commit 上写着自己的名字。我们还携带着组织对齐——清楚团队在努力实现什么目标，哪些技术债出于业务原因被容忍，在这个特定上下文中「好」是什么样子。我们以小步伐、以人类的节奏前进，这种节奏创造了让经验被触发和应用的思考空间。

Coding Agent 没有任何这些：没有社会责任感，对 300 行函数没有审美上的厌恶，没有「我们这里不这么做」的直觉，也没有组织记忆。它不知道哪个规范是承重墙、哪个只是习惯，也不知道技术上正确的方案是否符合团队正在努力实现的目标。

Harness 是将人类开发者经验外显化和明确化的尝试，但它只能走这么远。构建一套连贯的 Guides、Sensors 和自我纠正回路的成本很高，因此我们必须带着清晰目标优先排序：好的 harness 不一定要完全消除人类输入，而是要把人类输入引导到我们最重要的地方。

这最具哲学深度的部分，也是最容易被工具导向的讨论跳过的部分。Böckeler 在这里做了一件重要的事：她没有宣称 harness 能够替代人类判断，而是精确地划定了边界——harness 是人类经验的外显化，但人类经验的本质是「在特定社会和组织语境中形成的判断力」，这是无法被完全 Coding 的。这个判断意味着：**harness 工程的终点不是「无需人工监督的 agent」，而是「把人工监督用在刀刃上的 agent」**。这两个目标听起来相似，但对 harness 的设计哲学有根本性的影响。

八、业界实践：真实案例印证

原文列举了几个来自一线团队的实际案例，值得仔细看：

OpenAI：记录了他们的 harness 架构——由自定义 linter 和结构测试强制执行的分层架构，以及定期「垃圾回收」机制：扫描漂移并让 agent 提出修复建议。他们的结论是：**「我们目前最困难的挑战集中在如何设计环境、反馈回路和控制系统上。」**

Stripe：在其「minions」项目中描述了类似实践——基于启发式的 pre-push hook，按需运

行相关 linter，强调「把反馈左移」的重要性，并通过「blueprints」将反馈 sensor 集成到 agent 工作流程中。

Thoughtworks 团队：正在用计算型和推理型 Sensors 的组合应对架构漂移——例如用 agent 和自定义 linter 的混合提升 API 质量，或用「清洁工军团」提升代码质量。

这些案例的共同指向是：**最难的工程问题不再是让模型更聪明，而是设计让模型可靠工作的环境。**

九、尚未解决的问题

原文坦诚地列出了当前 harness 工程仍待解决的开放问题，这些问题的严肃性，正是这篇文章的价值所在：

随着 harness 增长，如何保持 Guides 和 Sensors 的内部一致性，避免相互矛盾？

当指令和反馈信号指向不同方向时，我们能在多大程度上信任 agent 做出合理取舍？

如果 Sensors 从不触发，这是质量高的标志，还是检测机制不足的标志？

我们需要类似代码覆盖率和 mutation testing 那样的工具，来评估 harness 的覆盖范围和质量。

目前前馈和反馈控制散落在各个交付步骤中，存在工具整合的空间，能帮助我们将它们作为一个系统来配置、同步和推理。

构建外层 harness 正在成为一种持续的工程实践，而不是一次性的配置。

写在最后

这篇文章发表在 Martin Fowler 的个人网站上，不是偶然——Fowler 的网站历来是软件工程领域概念走向成熟的「认证场所」。Harness Engineering 从一个硅谷小圈子的讨论词汇，进入这里，意味着它已经足够严肃，值得整个行业系统性地对待。

Böckeler 这篇文章最大的价值，不是提供了一套现成答案，而是提供了一套**精确的语言和分析框架**——让我们能够在更高的层次上讨论 agent 工程：不是「用了什么工具」，而是「设计了怎样的控制系统」。

这个框架本身，或许就是你构建第一套 harness 最重要的起点。